SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192324088
Name	Shaik Ashraf

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.

(BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I Shaik Ashraf / 192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Question 1 – Uneven Input Splits and Idle Reducers

Objective

Analyze why a Hadoop job repeatedly fails when input splits are uneven and reducers remain idle, then propose practical corrective actions.

Key Constraints

- Input data is distributed unevenly across input splits.
- Some mapper tasks take much longer than others, creating a straggler effect.
- Reducers remain idle because the map side does not produce balanced intermediate data at the required rate.
- The job must become more balanced without unnecessarily increasing cluster resources.

Identified Error

The most likely design error is poor input partitioning combined with data skew. Oversized or uneven input files/splits can cause a small number of mapper tasks to process disproportionately large amounts of data. Reducers then wait for mapper output, so cluster resources are underutilized and the job may exceed its execution window or fail.

Root Causes

- Unequal input file sizes or unsuitable split sizing.
- Many small files creating inefficient task distribution.
- Skewed records causing some map tasks to perform substantially more work.
- Poor partitioning of intermediate keys can also overload specific reducers.
- Insufficient monitoring of task duration and data distribution.

Corrective Actions

- Use suitable HDFS block and input-split sizing so work is divided more evenly.
- Combine very small files where appropriate to reduce excessive mapper overhead.
- Use input formats and partitioning logic that distribute records more uniformly.
- Apply a Combiner when the aggregation is associative and commutative to reduce shuffle volume.
- Use a custom partitioner or salting strategy when key skew creates reducer hotspots.
- Monitor mapper and reducer execution times and identify straggler tasks.
- Adjust mapper/reducer counts only after correcting the underlying data-distribution problem.

Error Analysis Summary

Problem Area	Likely Error	Corrective Action
Input Splits	Uneven split sizes	Tune split/block sizing and file layout.

Small Files	Too many tiny files	Merge/compact small files where appropriate.
Mapper Work	Data skew	Redistribute or preprocess skewed records.
Shuffle	Large intermediate output	Use Combiner and early filtering.
Reducer Load	Hot keys / uneven partitions	Use balanced partitioning or a custom partitioner.

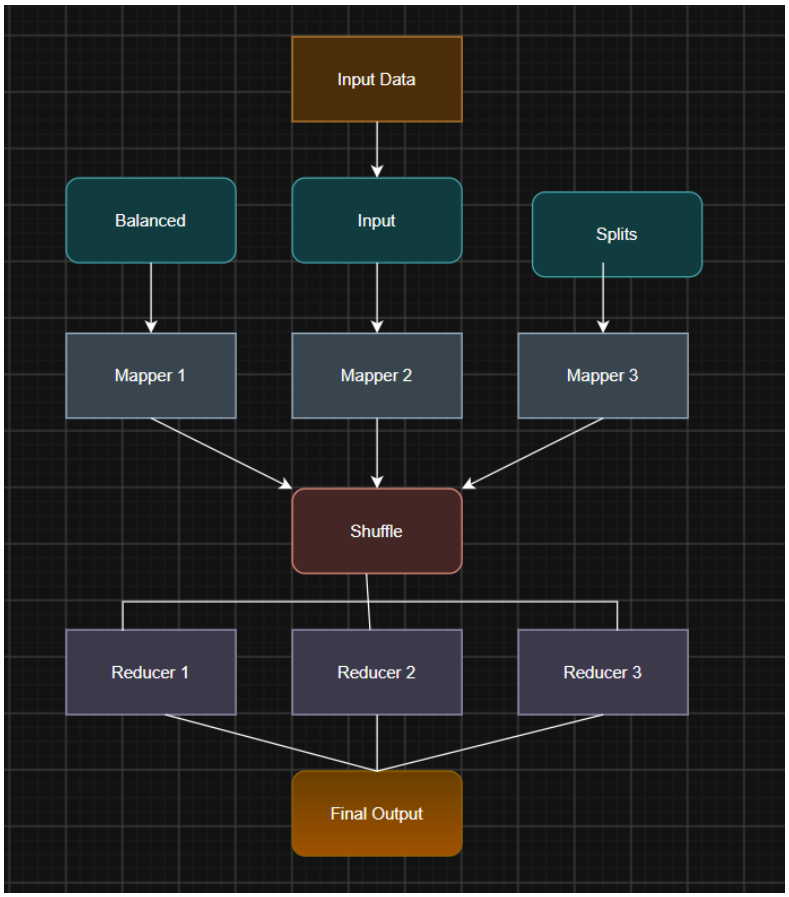
Technical Justification

Hadoop input processing is fundamentally parallel. If the input is split poorly, the theoretical parallelism of the cluster is not converted into useful throughput. A balanced split strategy reduces straggler tasks, while filtering and local aggregation reduce the amount of data sent through the shuffle phase.

Error Analysis Flowchart

Input Data in HDFS
↓
↓
↓
Balanced Input Splits
↓
↓
↓
Mapper Tasks in Parallel
↓
↓
↓
Filter + Parse Records
↓
↓
↓
Combiner
↓
↓
↓
Balanced Shuffle / Partition
↓
↓
↓
Reducers Process Data
↓

↓
↓
Final Output



Question 2 – HDFS Data Unavailability After a Node Failure

Objective

Analyze the probable HDFS replication or NameNode design error causing data unavailability after a single DataNode failure.

Key Constraints

- Data should remain accessible after a single node failure.
- HDFS must maintain sufficient block replicas.
- Metadata management must not create a single point of failure.
- The design should recover automatically or with minimal operational intervention.

Identified Error

The likely design error is inadequate block replication, poor replica placement, or a single-point-of-failure NameNode architecture. If important blocks have only one copy, a DataNode failure can make those blocks unavailable. If the NameNode has no high-availability arrangement, metadata-service failure can also make the HDFS namespace inaccessible even when DataNodes are healthy.

Root Causes

- Replication factor set too low for the required failure tolerance.
- Under-replicated blocks not detected or repaired promptly.
- Replicas placed in the same failure domain such as one rack.
- Single NameNode without a standby arrangement.
- Insufficient monitoring of DataNode health and block-replication status.

Corrective Actions

- Set an appropriate replication factor, such as 3 where the assessment design permits it.
- Enable rack awareness so replicas are distributed across failure domains.
- Use NameNode High Availability with Active and Standby NameNodes.
- Monitor under-replicated blocks and allow HDFS to re-replicate them on healthy DataNodes.
- Use health monitoring and alerts for DataNode failures.
- Verify recovery procedures through controlled failure testing.

Error Analysis Summary

Design Element	Probable Error	Corrective Action
Replication	Too few copies	Use replication factor appropriate to failure requirements.
Replica Placement	Copies in same rack/domain	Enable rack-aware

		placement.
NameNode	Single point of failure	Deploy Active + Standby NameNodes.
Monitoring	Under-replication unnoticed	Monitor block and node health.
Recovery	No tested failover	Test DataNode and NameNode failure scenarios.

Technical Justification

HDFS separates namespace management from block storage. DataNodes store blocks, while the NameNode manages metadata. Fault tolerance therefore requires both sufficient block replicas and a resilient metadata service. Rack-aware placement and NameNode high availability address different failure modes and should be designed together.

Error Analysis Flowchart

Client Requests Data
↓
↓
↓
NameNode Locates Block Replicas
↓
↓
↓
DataNode Failure Detected
↓
↓
↓
Healthy Replica Available
↓
↓
↓
HDFS Re-replication
↓
↓
↓
Standby NameNode Supports Metadata HA
↓
↓
↓
Data Remains Available

Question 3 – Duplicate Records in ETL and Apache NiFi Ingestion

Objective

Identify the likely causes of duplicate records created by an ETL and Apache NiFi ingestion workflow and recommend corrective measures.

Key Constraints

- Each source record should be represented correctly in the analytics store.
- Retries should not create additional copies.
- Multiple ingestion components may process the same source data.
- Data lineage and processing status should remain traceable.

Identified Error

The likely design error is a non-idempotent ingestion process. When NiFi or an ETL job retries a failed transfer, or when the same source batch is picked up by multiple flows, the destination may insert the record again because there is no reliable unique identifier, deduplication rule, or transaction/checkpoint mechanism.

Root Causes

- Repeated retries without idempotent destination writes.
- Missing unique business key or source record identifier.
- Overlapping NiFi schedules or duplicate flow paths.
- ETL and NiFi both ingesting the same source data independently.
- Missing batch ID, checksum, timestamp, or ingestion-status metadata.
- No downstream uniqueness constraint or deduplication step.

Corrective Actions

- Assign a stable source ID or business key to each record.
- Use batch IDs and ingestion timestamps to track processing state.
- Use checksums or hashes for file-level duplicate detection.
- Make destination writes idempotent so the same batch can be safely retried.
- Add a deduplication stage before loading the analytics store.
- Ensure NiFi flow schedules and ETL schedules do not overlap for the same source.
- Use provenance and processing-status metadata to trace every FlowFile/batch.

Error Analysis Summary

Cause	Effect	Recommended Fix
Non-idempotent retry	Same record inserted again	Use idempotent upsert/merge logic.
Missing unique key	Duplicates cannot be reliably detected	Create stable source/business key.

Overlapping flows	Same source processed twice	Coordinate NiFi and ETL schedules.
Missing metadata	Processing history is unclear	Track batch ID, source, timestamp and status.
No deduplication	Duplicate records reach analytics	Add validation/deduplication before final load.

Technical Justification

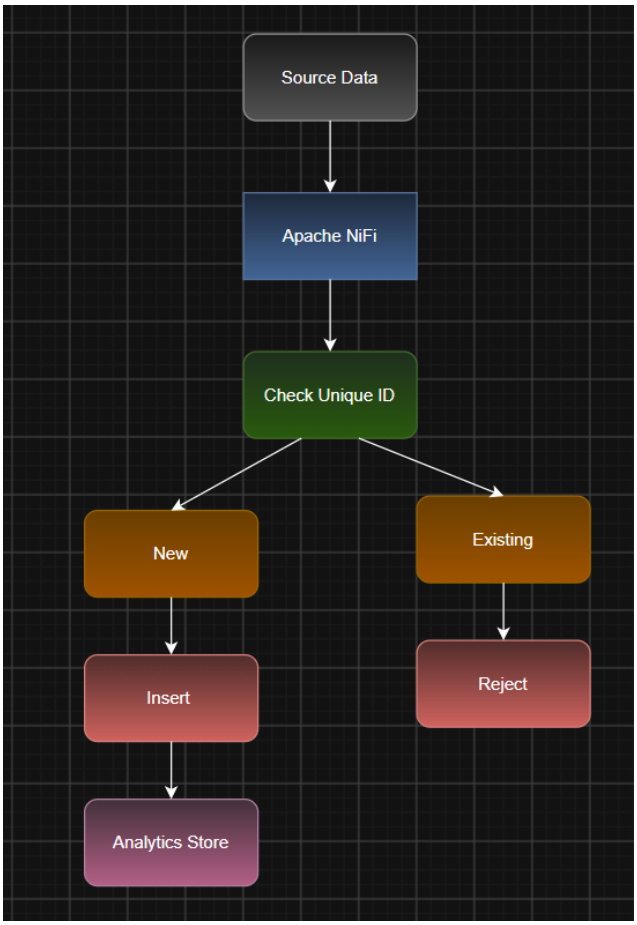
A reliable ingestion system must be idempotent: repeating the same operation should not create a different logical result. Stable identifiers, batch metadata, deduplication and coordinated scheduling make retries safe and provide traceability across NiFi and ETL stages.

Error Analysis Flowchart

Source Data
↓
↓
↓
NiFi / ETL Ingestion
↓
↓
↓
Assign Source ID + Batch ID
↓
↓
↓
Validate + Checksum
↓
↓
↓
Duplicate Detection
↓
↓
↓
Idempotent Load / Upsert
↓
↓
↓
Analytics Store
↓
↓

↓

Provenance + Monitoring



Question 4 – YARN Resource Starvation

Objective

Analyze the scheduling error that causes resource starvation when multiple users submit jobs and propose a better YARN resource-management approach.

Key Constraints

- Multiple users must share cluster resources fairly.
- One user or application should not monopolize CPU or memory.
- Important workloads may require controlled priority.
- Resource allocation should remain predictable as concurrent submissions increase.

Identified Error

The likely scheduling error is an uncontrolled or poorly configured resource-allocation policy. A queue without capacity limits, fair sharing, application limits, or suitable priorities can allow a few large applications to consume most cluster resources, causing other users' jobs to wait indefinitely.

Root Causes

- Single queue with no capacity or fair-share configuration.
- No per-user or per-application resource limits.
- Large applications requesting excessive containers.
- Improper priority settings causing starvation of other workloads.
- Insufficient queue-level isolation and monitoring.

Corrective Actions

- Create logical YARN queues for different workload classes or user groups.
- Use Capacity Scheduler or Fair Scheduler principles to allocate resources predictably.
- Set minimum and maximum queue capacities.
- Apply user and application limits to prevent resource monopolization.
- Give critical ETL workloads controlled priority while preserving minimum capacity for other queues.
- Monitor queue utilization, pending applications, and container allocation.
- Use ResourceManager high availability where required for resilient resource management.

Error Analysis Summary

Resource Problem	Scheduling Error	Better Approach
Queue starvation	No guaranteed capacity	Set queue minimum capacities.
User monopolization	No user/application limits	Configure user and

		application limits.
Priority abuse	Uncontrolled priorities	Use controlled priority and fair sharing.
Mixed workloads	No isolation	Create workload-specific queues.
Poor visibility	No monitoring	Track queue utilization and pending jobs.

Technical Justification

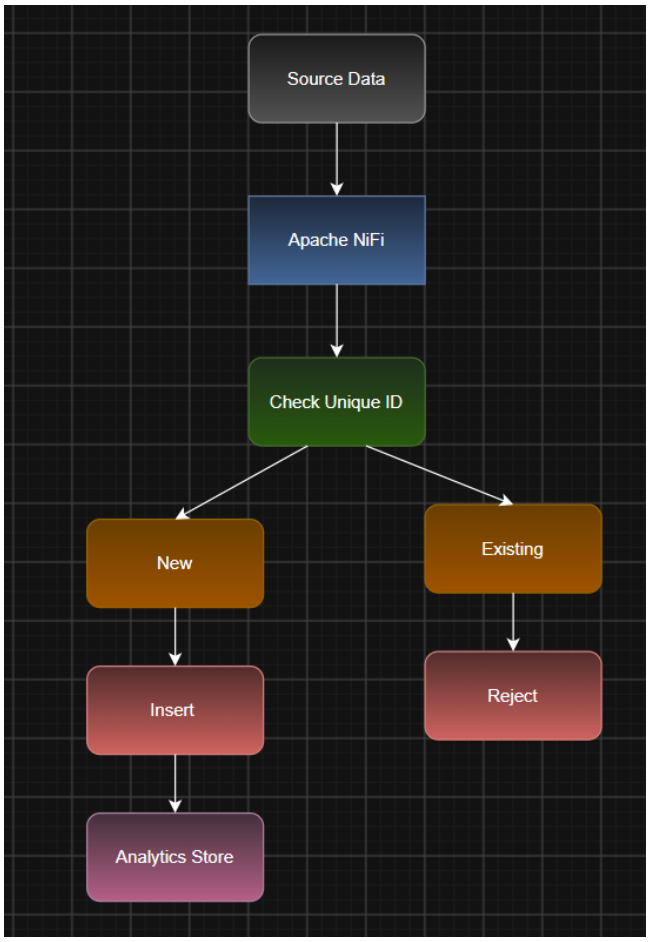
YARN separates resource management from application execution. Queue-based scheduling, capacity/fair-share policies and application limits prevent resource starvation. Controlled priority can be used for important workloads without allowing them to permanently monopolize the cluster.

Error Analysis Flowchart

Multiple User Job Submissions
↓
↓
↓
YARN ResourceManager
↓
↓
↓
Queue Classification
↓
↓
↓
Capacity / Fair-Share Allocation
↓
↓
↓
User + Application Limits
↓
↓
↓
Containers on NodeManagers
↓
↓
↓
Jobs Execute Concurrently
↓
↓

↓

Monitor and Rebalance



Question 5 – Outdated Data After Distributed Storage Recovery

Objective

Identify the probable replication or backup design flaw that causes recovery to restore outdated data and recommend a reliable approach.

Key Constraints

- Recovery should restore the most recent valid data available under the recovery policy.
- Replication and backup copies must be synchronized or version-aware.
- Recovery procedures must preserve data consistency.
- Backup restoration should be tested rather than assumed to work.

Identified Error

The probable design flaw is an outdated or poorly coordinated backup/replication strategy. A backup copy may have been created before the latest changes, or replication may have lagged behind the primary data. If recovery blindly restores the oldest available copy without checking version, timestamp, or recovery point, outdated data can replace newer data.

Root Causes

- Backups run too infrequently for the required recovery point objective.
- Replication lag is not monitored.
- Old snapshots are restored without validating their timestamps or versions.
- Backup metadata does not clearly identify the latest successful backup.
- Restore procedures are not regularly tested.
- No clear Recovery Point Objective (RPO) or retention policy.

Corrective Actions

- Define an explicit RPO and Recovery Time Objective (RTO).
- Use scheduled backups at intervals consistent with the required RPO.
- Track backup completion, timestamps, versions, and integrity checks.
- Monitor replication lag and alert when replicas fall behind.
- Use version-aware recovery so the latest valid recovery point is selected.
- Maintain multiple recovery points according to retention policy.
- Perform periodic restore tests to verify that backups are usable and current.

Error Analysis Summary

Issue	Probable Flaw	Corrective Action
Old backup restored	Backup schedule too infrequent	Increase backup frequency to meet RPO.
Replication lag	Lag not monitored	Monitor and alert on

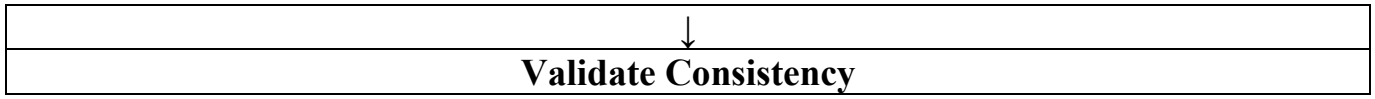
		replication delay.
Wrong recovery point	No version/timestamp validation	Select latest valid recovery point.
Unverified backups	Restore never tested	Perform scheduled recovery tests.
Unclear policy	No RPO/retention definition	Define RPO, RTO and retention rules.

Technical Justification

Replication and backup are not identical. Replication improves availability and protects against certain failures, while backups provide historical recovery points. A reliable recovery design therefore needs versioned recovery points, defined RPO/RTO values, integrity checks and tested restoration procedures.

Error Analysis Flowchart

Primary Distributed Data
↓
↓
↓
Replication + Scheduled Backup
↓
↓
↓
Version / Timestamp Tracking
↓
↓
↓
Backup Integrity Verification
↓
↓
↓
Failure / Recovery Event
↓
↓
↓
Select Latest Valid Recovery Point
↓
↓
↓
Restore Data
↓
↓


↓
Validate Consistency

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Submission Checklist

- All five error-analysis questions are answered.
- Each answer clearly separates the identified error, root causes and corrective actions.
- Hadoop, HDFS, MapReduce, YARN, ETL and Apache NiFi concepts are used where relevant.
- Each question ends with a vertical flowchart summarizing the proposed analysis/correction.
- The document maintains consistent academic formatting and terminology.